



Pashov Audit Group

Regnum Aurum Security Review

November 27th 2025 - December 14th 2025



Contents

1. About Pashov Audit Group	3
2. Disclaimer	3
3. Risk Classification	3
4. About Regnum Aurum	4
5. Executive Summary	4
6. Findings	6
Medium findings	8
[M-01] User debt and global borrow divergence distorts usage and rates	8
[M-02] Collecting protocol fee results in inflated <code>reserve.totalLiquidity</code>	10
[M-03] Vault rounding losses enable principal dilution	13
[M-04] Updating indexes with new rates causes inaccurate interest accrual	15
Low findings	18
[L-01] <code>getNormalizedDebt</code> returns increasing usage index when there are no borrows	18
[L-02] <code>reserve.usageIndex</code> can increase when there are no borrows	20
[L-03] Protocol tokens do not validate scaled amounts	20
[L-04] Burned <code>RAACNFT</code> tokens can be re-minted	22
[L-05] Liquidations fail when <code>LendingPool</code> is paused	23
[L-06] <code>RToken</code> does not include interest in <code>Mint</code> event emission	24
[L-07] Prime rate change limits are too permissive and prevent decreases below 5%	25
[L-08] <code>LendingPool.updateState()</code> does not recalculate interest rates after accrual	26
[L-09] Missing event emission for key parameter changes	26
[L-10] Liquidity buffer ratio must initialize to zero as per docs	26
[L-11] Min price <code>Threshold</code> lets NFT minting at discount during depeg	27



1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Twitter [@pashovkrum](#).

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive



4. About Regnum Aurum

Regnum Aurum (RAAC) is a fractionalization platform that tokenizes real estate into NFTs (RAACNFT) and fractional index tokens (iRAAC), enabling on-chain lending, borrowing, and liquidity against property value. By combining Chainlink-powered appraisals, a hybrid RWA Vault, and veRAAC governance the protocol enables programmable debt positions against real estate assets with on-chain liquidation mechanisms.

5. Executive Summary

A time-boxed security review of the **RegnumAurumAcquisitionCorp/contracts** repository was done by Pashov Audit Group, during which **Hals**, **JCN**, **merlinboii**, **BenRai** engaged to review **Regnum Aurum**. A total of **15** issues were uncovered.

Protocol Summary

Project Name	Regnum Aurum
Protocol Type	RWA tokenization
Timeline	November 27th 2025 - December 14th 2025

Review commit hash:

- [0d7710d0e4cee3f17bcd3964f129cb520990e435](#)
(RegnumAurumAcquisitionCorp/contracts)

Fixes review commit hash:

- [60cde54a1402d20a05fe219ff085c01db5bab1d3](#)
(RegnumAurumAcquisitionCorp/contracts)

Scope

ERC20Collector.sol FeeCollector.sol NFTRoyaltyFeeCollector.sol
RAACTokenCollector.sol RWAIndexTokenCollector.sol Treasury.sol
BaseChainlinkFunctionsOracle.sol BaseVRFv2Consumer.sol crvUSDPriceOracle.sol
CrvUSDToUSDOracle.sol RAACHousePriceOracle.sol RAACPrimeRateOracle.sol
RWAIndexTokenOracle.sol ZkMeKYCVerifyModule.sol KYCVerifyModule.sol
ERC20AssetAdapter.sol ERC721AssetAdapter.sol LendingPool.sol
LendingPoolStorage.sol LiquidationProxy.sol VaultProxy.sol
LiquidationStrategyProxy.sol LiquidationSwap.sol StabilityPool.sol
StabilityPoolStorage.sol ComplianceRegistry.sol RAACHousePrices.sol
WithCompliance.sol RWAIndexToken.sol RToken.sol RAACNFT.sol DEToken.sol
DebtToken.sol ERC20VaultAdapter.sol ERC721VaultAdapter.sol



RAACNFTVaultAdapterOracle.sol

RAACNFTVaultAdapterV2.sol

RWAVault.sol

PercentageMath.sol

TimeWeightedAverage.sol

WadRayMath.sol

ReserveLibrary.sol

StringUtils.sol



6. Findings

Findings count

Severity	Amount
Medium	4
Low	11
Total findings	15

Summary of findings

ID	Title	Severity	Status
[M-01]	User debt and global borrow divergence distorts usage and rates	Medium	Resolved
[M-02]	Collecting protocol fee results in inflated <code>reserve.totalLiquidity</code>	Medium	Resolved
[M-03]	Vault rounding losses enable principal dilution	Medium	Resolved
[M-04]	Updating indexes with new rates causes inaccurate interest accrual	Medium	Resolved
[L-01]	<code>getNormalizedDebt</code> returns increasing usage index when there are no borrows	Low	Resolved
[L-02]	<code>reserve.usageIndex</code> can increase when there are no borrows	Low	Resolved
[L-03]	Protocol tokens do not validate scaled amounts	Low	Acknowledged
[L-04]	Burned <code>RAACNFT</code> tokens can be re-minted	Low	Resolved
[L-05]	Liquidations fail when <code>LendingPool</code> is paused	Low	Resolved
[L-06]	<code>RToken</code> does not include interest in <code>Mint</code> event emission	Low	Resolved
[L-07]	Prime rate change limits are too permissive and prevent decreases below 5%	Low	Resolved
[L-08]	<code>LendingPool.updateState()</code> does not recalculate interest rates after accrual	Low	Resolved
[L-09]	Missing event emission for key parameter changes	Low	Resolved



ID	Title	Severity	Status
[L-10]	Liquidity buffer ratio must initialize to zero as per docs	Low	Resolved
[L-11]	Min price <code>Threshold</code> lets NFT minting at discount during depeg	Low	Acknowledged



Medium findings

[M-01] User debt and global borrow divergence distorts usage and rates

Severity

Impact: Medium

Likelihood: Medium

Description

The protocol applies different rounding strategies when accounting for user-level debt versus global borrow totals. User debt is repeatedly rounded up at multiple points (per-position scaling, interest accrual, aggregation across positions), while the global `_rawTotalBorrows` tracks principal-only values and is adjusted proportionally during burns.

```
// @LendingPool._positionScaledDebt()
function _positionScaledDebt(CollateralPosition memory position) internal view returns
(uint256) {
    uint256 usageIndex = ReserveLibrary.getUsageIndex(reserve, rateData);

    if (position.positionIndex == 0 || position.rawDebtBalance == 0) {
        return position.rawDebtBalance;
    }

    // All scaling uses : raw * (usageIndex / positionIndex)
@787> uint256 indexMultiplier = usageIndex.rayDiv(position.positionIndex); //@note : rounds-up
@788> return position.rawDebtBalance.rayMul(indexMultiplier); //@note : rounds-up
}
```

```
// @DebtToken.totalSupply()
function totalSupply() public view override(ERC20, IERC20) returns (uint256) {
@197> return _rawTotalBorrows.rayMul(ILendingPool(_lendingPool).getNormalizedDebt()); //
@note : rounds-up
}
```

Over time, this asymmetry causes a cumulative drift: individual users' scaled debt can grow slightly faster than what is implied by `_rawTotalBorrows * usageIndex`. During repayment, the burn logic reduces `_rawTotalBorrows` based on the ratio between the user's scaled repayment amount and the global scaled total supply. Because the user-side value includes rounding-up effects that are not symmetrically reflected in `_rawTotalBorrows`, the proportional reduction can remove more raw borrows than intended.

```
// @DebtToken.burn()
function burn(
    address from,
    uint256 amount,
```



```
uint256 index,
bytes calldata asset
) external onlyLendingPool returns (uint256, uint256, uint256, uint256) {
    //...

    (uint256 positionIndex, ) = ILendingPool(_lendingPool).getPositionData(adapter, from,
data);

    uint256 userIncrease = _mintUserInterest(from,
ILendingPool(_lendingPool).getPositionDebt(adapter, from, data),
ILendingPool(_lendingPool).getPositionScaledDebt(adapter, from, data));

    // Burn raw amount of debt token from user (will get the mintInterest executed via
update to == address(0))
@167> uint256 totalSupplyBeforeBurn = totalSupply();
    _burn(from, scaledAmount.toUint128());

@170> uint256 percentageDrop = scaledAmount * 1e18 / totalSupplyBeforeBurn;
@171> _rawTotalBorrows -= _rawTotalBorrows * percentageDrop / 1e18;

    emit Burn(from, scaledAmount, index, positionIndex);

    return (scaledAmount, totalSupply(), scaledAmount, userIncrease);
}
```

This drift is not necessarily visible in a single borrow/repay cycle, but it compounds over time as users interact with the system. Eventually, this can lead to inconsistencies between per-position debt and global borrow accounting, increasing the risk of failures in subsequent repayments of latest borrowers in the system.

The effect does not remain local to the `DebtToken` : the protocol explicitly derives `reserve.totalUsage` from `DebtToken.totalSupply()` , which is then used to compute utilization, borrow rates, supply rates, index growth, and protocol fee accrual:

```
// @ReserveLibrary.updateInterestRatesAndLiquidity()
function updateInterestRatesAndLiquidity(ReserveData storage reserve, ReserveRateData storage
rateData, uint256 liquidityAdded, uint256 liquidityTaken) internal {
    //...

    // Update total usage with the amount of debt taken
@218> reserve.totalUsage = IDebtToken(reserve.reserveDebtTokenAddress).totalSupply();

    // Calculate utilization rate
@221> uint256 utilizationRate = calculateUtilizationRate(reserve.totalLiquidity,
reserve.totalUsage);
    rateData.currentUtilizationRate = utilizationRate;
    //...
    (uint256 calculatedLiquidityRate, uint256 calculatedProtocolFeeRate) =
calculateLiquidityRate(
    utilizationRate,
    rateData.currentUsageRate,
    rateData.protocolFeeRate,
@239> reserve.totalUsage
```



```
);  
//...  
}
```

As a result, any cumulative drift between per-position debt and `_rawTotalBorrows * usageIndex` directly propagates to system-wide parameters. Over time, this can lead to incorrect interest rate calculations, skewed index growth, and protocol fees that do not accurately reflect the true economic debt in the system.

Recommendations

Make mint and burn update `_rawTotalBorrows` using the same unit conversion and rounding path (convert between scaled debt and raw principal consistently), and cap the reduction applied to `_rawTotalBorrows` during burns to never revert.

[M-02] Collecting protocol fee results in inflated `reserve.totalLiquidity`

Severity

Impact: Low

Likelihood: High

Description

When liquidity enters the pool, either via deposits or repayments, the `reserve.totalLiquidity` internal state variable is incremented. When liquidity exits the pool, via withdrawals or borrows, the `reserve.totalLiquidity` is decremented:

```
function updateInterestRatesAndLiquidity(ReserveData storage reserve, ReserveRateData  
storage rateData, uint256 liquidityAdded, uint256 liquidityTaken) internal {  
    // Update total liquidity  
    if (liquidityAdded > 0) {  
        reserve.totalLiquidity = reserve.totalLiquidity + liquidityAdded.toUint128();  
    }  
    if (liquidityTaken > 0) {  
        if (reserve.totalLiquidity < liquidityTaken) revert InsufficientLiquidity();  
        reserve.totalLiquidity = reserve.totalLiquidity - liquidityTaken.toUint128();  
    }  
}
```

However, when the protocol fees are collected the `reserve.totalLiquidity` is not updated, even though this fee amount is transferred out of the `RToken` (removed from liquidity):

```
function claimProtocolFees(ReserveLibrary.ReserveData storage reserve, address  
feeCollector) external returns (address tokenAddress, uint256 claimedAmount) {  
    if (address(feeCollector) == address(0)) revert AddressCannotBeZero();  
  
    uint256 pendingAmount = reserve.pendingProtocolFeeAmount;  
    address reserveAssetAddress = reserve.reserveAssetAddress;
```



```
address reserveRTokenAddress = reserve.reserveRTokenAddress;
if(pendingAmount == 0) return (reserveAssetAddress, 0);

tokenAddress = reserveAssetAddress;
claimedAmount = pendingAmount;

reserve.pendingProtocolFeeAmount = 0;

IRToken(reserveRTokenAddress).transferAsset(address(this), pendingAmount);
bool approved = IERC20(reserveAssetAddress).approve(feeCollector, pendingAmount);
if (!approved) revert ApprovalFailed();
IBaseCollector(feeCollector).collectFee(reserveAssetAddress, address(this),
pendingAmount, keccak256("PROTOCOL_FEE"));
}
```

This means that the `reserve.totalLiquidity` will never be decremented by the collected protocol fees and will therefore be inflated (will be larger than the actual available liquidity). Overtime this will translate to the calculated `utilizationRate` being lower than intended, which will result in the liquidity rate being lower than intended (suppliers will earn less interest):

```
function updateInterestRatesAndLiquidity(ReserveData storage reserve, ReserveRateData
storage rateData, uint256 liquidityAdded, uint256 liquidityTaken) internal {
...
    // Calculate utilization rate
    uint256 utilizationRate = calculateUtilizationRate(reserve.totalLiquidity,
reserve.totalUsage);
    rateData.currentUtilizationRate = utilizationRate;

    // Update current usage rate (borrow rate)
    rateData.currentUsageRate = calculateBorrowRate(
        rateData.primeRate,
        rateData.baseRate,
        rateData.optimalRate,
        rateData.maxRate,
        rateData.optimalUtilizationRate,
        utilizationRate
    );

    // Update current liquidity rate
    (uint256 calculatedLiquidityRate, uint256 calculatedProtocolFeeRate) =
calculateLiquidityRate(
    utilizationRate,
    rateData.currentUsageRate,
    rateData.protocolFeeRate,
    reserve.totalUsage
);

    rateData.currentLiquidityRate = calculatedLiquidityRate;
    rateData.currentProtocolFeeRate = calculatedProtocolFeeRate;

...

    function calculateUtilizationRate(uint256 totalLiquidity, uint256 totalDebt) internal pure
returns (uint256) {
        if (totalLiquidity < 1) {
```



```
        return WadRayMath.RAY; // 100% utilization if no liquidity
    }
    uint256 utilizationRate = totalDebt.rayDiv(totalLiquidity + totalDebt).toUint128();
    return utilizationRate;
}
```

Contrast this approach with Aave's approach. Aave tracks fee shares for the protocol and then mints those fee shares to the protocol at a later time. After this point, the protocol is able to redeem those `ATokens` for underlying assets as normal users do, which will result in the internal virtual balance (equivalent to `reserve.totalLiquidity` here) decreasing normally. This mechanic does not exist in RAAC.

Another impact of an inflated `reserve.totalLiquidity` is that less liquidity will be allocated to the vault over time during rebalancing, which results in more idle liquidity and therefore less vault rewards for the protocol.

```
function rebalanceLiquidity() external onlyProxy {
    // if vault is not set, do nothing as this can be desired to disable vault
    if (address(vault) == address(0)) {
        return;
    }

    uint256 totalDeposits = reserve.totalLiquidity; // Total liquidity in the system
    uint256 desiredBuffer = totalDeposits.percentMul(parameters.liquidityBufferRatio);
    uint256 currentBuffer =
IERC20(reserve.reserveAssetAddress).balanceOf(reserve.reserveRTokenAddress);

    // Only deposit excess into vault if we're not in the middle of a withdrawal
    if (currentBuffer > desiredBuffer) {
        uint256 excess = currentBuffer - desiredBuffer;
        // Deposit excess into the vault
        depositIntoVault(excess);
    } else if (currentBuffer < desiredBuffer) {
        uint256 shortage = desiredBuffer - currentBuffer;
        // Check how much we can actually withdraw
        uint256 vaultBalance = _maxWithdraw(address(this));
        uint256 withdrawAmount = shortage > vaultBalance ? vaultBalance : shortage;
        if (withdrawAmount > 0) {
            // Withdraw what we can from the vault
            withdrawFromVault(withdrawAmount);
        }
    }
}
```

Recommendation

Update the `reserve.totalLiquidity` when the protocol fees are removed from the `RToken`.



[M-03] Vault rounding losses enable principal dilution

Severity

Impact: Medium

Likelihood: Medium

Description

The Lending Pool integrates with an external vault and automatically rebalances liquidity at each pool operation based on a configured buffer ratio. However, the actual total liquidity (idle balance + vault balance) can fall below the tracked `reserve.totalLiquidity` due to vault operations that systematically favor the vault and how yields are harvested.

There are multiple issues:

1. **Deposit rounding loss:** When depositing to the vault, shares received are rounded down, causing the actual withdrawable value to be less than the deposited amount immediately.

[VaultProxy.sol#L172-L200](#)

```
function depositIntoVault(uint256 amount) internal {
    // --- SNIPPED ---

    // Then approve and deposit into the vault and get shares in return
    IERC20(reserve.reserveAssetAddress).approve(address(vault), amount);
    @> uint256 shares = _deposit(amount, address(this));
    if (shares == 0) return;

    // Update state
    @> vaultRewards.totalDeposits += amount;
    vaultRewards.totalShares += shares;
    vaultRewards.lastDepositTime = block.timestamp;
}
```

1. **Withdrawal rounding loss:** When withdrawing from the vault, shares burned are rounded up, reducing more shares worth of assets than the requested withdrawal amount.
2. **Price changes treated as pure yield:** The `totalYield` calculation treats all `pricePerShare` increases as pure yield. When withdrawing the calculated yield amount, the actual remaining asset value can make it further fall below the tracked `totalDeposits`.
3. [VaultProxy.sol#L140-L141](#)
4. [VaultProxy.sol#L216-L217](#)

```
uint256 priceDifference = currentSharePrice - vaultRewards.lastSharePrice;
uint256 totalYield = (vaultRewards.totalShares * priceDifference) / 1e18;
```



From the root causes above, it allows attacker strategically withdraw improperly from the vault, leading to principal dilution and liquidity accounting discrepancies.

Consider the following attack vectors: 1. **Griefing by withdrawing dust amounts:** - Withdraw dust amounts (1-20 wei) to trigger vault rebalancing. - The vault overestimates shares to burn for the requested dust withdrawal assets.

1. **Griefing to DoS vault update:**
2. When vault update there is a need to withdraw all assets from the vault to update the vault and require to cover the tracked `totalDeposits`.

[LendingPool.sol#L1026-L1042](#)

```
function setVault(address newVault) external onlyOwner {
    // If we change the vault, we need to claim all rewards from the old vault
    if (address(vault) != address(newVault) && address(vault) != address(0)) {
@> (bool success, bytes memory returndata) = vaultProxy.delegatecall(
        abi.encodeWithSignature("withdrawAll()")
    );
    _handleProxyError(success, returndata);
    }
    // --- SNIPPED ---

    emit VaultUpdated(oldVault, newVault);
}
```

- Although the `withdrawAll()` will attempt to separate yield from principal after withdraw all the balance, user can front-run the `withdrawAll()` with withdraw dust amounts to trigger vault rebalancing and harvest the yields first.

[VaultProxy.sol#L70-L123](#)

```
function withdrawAll() external onlyProxy {
    // --- SNIPPED ---
@> uint256 maxWithdrawable = _maxWithdraw(address(this));

    if (maxWithdrawable < vaultRewards.totalDeposits) {
        revert("Not enough withdrawable amount to withdraw all");
    }

    // withdraw everything from the vault
    // --- SNIPPED --- : withdrawal logic

@> if (totalWithdrawn < vaultRewards.totalDeposits) {
        revert("Not enough withdrawable amount to withdraw all");
    }

    // Send the total deposit amount to the RToken. It will fail if the withdraw amount is not
    // enough.
    IERC20(reserve.reserveAssetAddress).safeTransfer(reserve.reserveRTokenAddress,
        vaultRewards.totalDeposits);

    // Calculate the total yield collected //@audit after withdraw all the balance
@> uint256 totalYield = totalWithdrawn - vaultRewards.totalDeposits;
```



```
if (totalYield > 0) {  
    vaultRewards.unclaimedRewards += totalYield;  
}  
  
// --- SNIPPED ---  
}
```

- Since no yield exists at `withdrawAll()` execution, the max withdrawable amount will only remain the diluted principal, which could be lesser than the tracked `totalDeposits`, leading to revert.

Proof of Concept: [here](#) - [test audit SCRVAULT diluteActualLiquidity stateProof](#) - [test audit SCRVAULT diluteActualLiquidity blockWithdrawAll](#)

Recommendation

While there is no single straightforward fix, revisit the potential risks and implementing a combination of the following mitigations would reduce the impact:

- The main approach is to ensure that the yields harvest does not dilute the principal.
- Consider applying minimum thresholds for vault rebalancing operations to prevent dust-amount withdrawal griefing attacks. Only trigger rebalancing when the amount exceeds a meaningful threshold.

[M-04] Updating indexes with new rates causes inaccurate interest accrual

Impact: Medium

Likelihood: Medium

Description

`LendingPool.setParameter()` calls `ReserveLibrary.updateInterestRatesAndLiquidity()` both before and after updating parameters. However, the function updates `reserve.liquidityIndex` and `reserve.usageIndex` using the new `currentLiquidityRate` and `currentUsageRate` for the entire elapsed `timeDelta`, rather than the rates that were actually in effect during that period.

This results in indexes being updated with rates that only became valid at the current block, leading to incorrect accrual of interest and protocol fees.

```
//File: contracts/core/pools/LendingPool.sol  
  
function setParameter(OwnerParameter param, uint256 newValue) external override {  
    // --- SNIPPED ---  
    @1> ReserveLibrary.updateInterestRatesAndLiquidity(reserve, rateData, 0, 0);  
    //
```



```
// --- SNIPPED : parameter updated here ---  
//  
@2> ReserveLibrary.updateInterestRatesAndLiquidity(reserve, rateData, 0, 0);  
}
```

At each call to `updateInterestRatesAndLiquidity`, the following happens:

Assume that `t` is the current block timestamp and `t-n` is the timestamp of the last update (where cached data is stored)

1. The function recalculates `currentLiquidityRate` and `currentUsageRate` using the up to time `t` parameters.

```
//File: contracts/libraries/pools/ReserveLibrary.sol  
  
function updateInterestRatesAndLiquidity(ReserveData storage reserve, ReserveRateData storage  
rateData, uint256 liquidityAdded, uint256 liquidityTaken) internal {  
    // --- SNIPPED : update total liquidity (no update in this case) ---  
  
    // Update total usage with the amount of debt taken  
    //@note  
    //> `_rawTotalBorrows` * `getUsageIndex()` --> recalculate the index using  
    //> `calculateBorrowRate(<cached data: t-n>`, `timeDelta: t - (t-n)`, cached  
    `reserve.usageIndex_t-n`  
    @1> reserve.totalUsage = IDebtToken(reserve.reserveDebtTokenAddress).totalSupply();  
  
    // Calculate utilization rate  
    @2> uint256 utilizationRate = calculateUtilizationRate(reserve.totalLiquidity,  
reserve.totalUsage);  
    rateData.currentUtilizationRate = utilizationRate;  
  
    // Update current usage rate (borrow rate)  
    @3> rateData.currentUsageRate = calculateBorrowRate(  
        // --- SNIPPED: cached rateData: primeRate, baseRate, optimalRate, maxRate,  
optimalUtilizationRate  
        ...,  
        utilizationRate //> from @2  
    );  
  
    // Update current liquidity rate  
    (uint256 calculatedLiquidityRate, uint256 calculatedProtocolFeeRate) = calculateLiquidityRate(  
        utilizationRate, //> from @2  
        rateData.currentUsageRate, //> from @3  
        rateData.protocolFeeRate,  
        reserve.totalUsage //> from @1  
    );  
  
    @4> rateData.currentLiquidityRate = calculatedLiquidityRate;  
    rateData.currentProtocolFeeRate = calculatedProtocolFeeRate;  
  
    // Update the reserve interests  
    @5> updateReserveInterests(reserve, rateData);
```



```
emit InterestRatesUpdated(rateData.currentLiquidityRate, rateData.currentUsageRate);  
}
```

1. It then updates `reserve.liquidityIndex` and `reserve.usageIndex` for the entire elapsed time since the last update (`t - (t-n)`), but uses these **new** rates for the whole period.

```
//File: contracts/libraries/pools/ReserveLibrary.sol  
  
function updateReserveInterests(ReserveData storage reserve, ReserveRateData storage rateData)  
internal {  
    // --- SNIPPED : timestamp and oldLiquidityIndex check ---  
  
    // Update liquidity index using linear interest  
    reserve.liquidityIndex = calculateLiquidityIndex(  
        rateData.currentLiquidityRate,    //> from @4  
        timeDelta,  
        oldLiquidityIndex  
    );  
  
    // Update usage index (debt index) using compounded interest  
    reserve.usageIndex = calculateUsageIndex(  
        rateData.currentUsageRate,        //> from @3  
        timeDelta,  
        reserve.usageIndex  
    );  
  
    // Update the last update timestamp  
    reserve.lastUpdateTimestamp = uint40(block.timestamp);  
  
    // --- SNIPPED : update the pending protocol fee amount (using the new  
    reserve.liquidityIndex) ---  
}
```

This is incorrect because the new rates should only apply from the current block onward. Using them for the entire elapsed period causes interest and protocol fee calculations to be based on values that were not actually in effect during that time, resulting in inaccurate accruals.

Recommendation

Update indexes using the old rates for the elapsed period first, then apply the parameter change and update the rate to be effective with the new config for the new period onwards.



Low findings

[L-01] `getNormalizedDebt` returns increasing usage index when there are no borrows

When the `LendingPool` is deployed, the `currentUtilizationRate` is initialized to `100%` :

```
constructor(  
    address _reserveAssetAddress,  
    address _rTokenAddress,  
    address _debtTokenAddress,  
    address _liquidationProxyAddress,  
    address _vaultProxy,  
    address _complianceRegistry,  
    uint256 _initialPrimeRate,  
    address _admin  
  
    ) Ownable(msg.sender) WithCompliance(_complianceRegistry) {  
...  
    rateData.currentUtilizationRate = WadRayMath.RAY;  
...  
}
```

This utilization rate is then used when dynamically calculating the `usageIndex` via `LendingPool::getNormalizedDebt` :

```
function getUsageIndex(ReserveData storage reserve, ReserveRateData storage rateData)  
internal view returns (uint256) {  
    uint256 timeDelta = block.timestamp - uint256(reserve.lastUpdateTimestamp);  
    if(timeDelta < 1) {  
        return reserve.usageIndex;  
    }  
  
    return calculateUsageIndex(  
        calculateBorrowRate(rateData.primeRate, rateData.baseRate, rateData.optimalRate,  
        rateData.maxRate, rateData.optimalUtilizationRate, rateData.currentUtilizationRate),  
        timeDelta,  
        reserve.usageIndex  
    );  
}
```

```
function calculateBorrowRate(  
    uint256 primeRate,  
    uint256 baseRate,  
    uint256 optimalRate,  
    uint256 maxRate,  
    uint256 optimalUtilizationRate,  
    uint256 utilizationRate  
) internal pure returns (uint256) {  
...  
    if (utilizationRate <= optimalUtilizationRate) {  
        rate = optimalRate;  
    }
```



```
    } else {
        uint256 excessUtilization = utilizationRate - optimalUtilizationRate;
        uint256 maxExcessUtilization = WadRayMath.RAY - optimalUtilizationRate;
        uint256 rateSlope = maxRate - optimalRate; // u = utilization penalty factor
        uint256 rateIncrease =
excessUtilization.rayMul(rateSlope).rayDiv(maxExcessUtilization);
        rate = optimalRate + rateIncrease;
        rate = rate > maxRate ? maxRate : rate;
    }
    return rate;
}
```

As we can see above, the borrow rate will be initially calculated as the `maxRate`, which will result in the `usageIndex` being reported as increasing at this rate. As a result, after `LendingPool` is deployed and until user's interact with the system, the `getNormalizedDebt` function will continue to return increased `usageIndex` as time goes on. This can mislead integrators and UIs that query this function after deployment.

Note that this is also the case when the system has no more liquidity and no more borrows, i.e. is empty. When the pool is emptied, the `updateInterestRatesAndLiquidity` function will be invoked, which will calculate and update the utilization as `100%` when there are no borrows:

```
function updateInterestRatesAndLiquidity(ReserveData storage reserve, ReserveRateData
storage rateData, uint256 liquidityAdded, uint256 liquidityTaken) internal {
    ...
    // Calculate utilization rate
    uint256 utilizationRate = calculateUtilizationRate(reserve.totalLiquidity,
reserve.totalUsage);
    rateData.currentUtilizationRate = utilizationRate;

    // Update current usage rate (borrow rate)
    rateData.currentUsageRate = calculateBorrowRate(
        rateData.primeRate,
        rateData.baseRate,
        rateData.optimalRate,
        rateData.maxRate,
        rateData.optimalUtilizationRate,
        utilizationRate
    );
}
```

```
function calculateUtilizationRate(uint256 totalLiquidity, uint256 totalDebt) internal pure
returns (uint256) {
    if (totalLiquidity < 1) {
        return WadRayMath.RAY; // 100% utilization if no liquidity
    }
    ...
}
```

This update is correct if there are borrows, but it will drastically inflate the utilization if there are no borrows. As a result, the `reserve.currentUsageRate` will be updated to the `maxRate`, and `getNormalizedDebt` will continue to incorrectly report increased `usageIndex` values.



Consider initializing the `currentUtilizationRate` to `0` and returning `0` in `calculateUtilizationRate` if there are no active borrows in the system.

[L-02] `reserve.usageIndex` can increase when there are no borrows

`ReserveLibrary::updateReserveInterests` increases the `reserve.usage` index without first checking if there are any borrows in the system:

```
function updateReserveInterests(ReserveData storage reserve, ReserveRateData storage
rateData) internal {
    uint256 timeDelta = block.timestamp - uint256(reserve.lastUpdateTimestamp);
    if (timeDelta < 1) {
        return;
    }

    uint256 oldLiquidityIndex = reserve.liquidityIndex;
    if (oldLiquidityIndex < 1) revert LiquidityIndexIsZero();

    // Update liquidity index using linear interest
    reserve.liquidityIndex = calculateLiquidityIndex(
        rateData.currentLiquidityRate,
        timeDelta,
        oldLiquidityIndex
    );

    // Update usage index (debt index) using compounded interest
    reserve.usageIndex = calculateUsageIndex(
        rateData.currentUsageRate,
        timeDelta,
        reserve.usageIndex
    );
}
```

As a result, an idle pool with no active borrows will continue to report debt interest accrued via the increased `usageIndex` state variable. This will result in view functions, such as `LendingPool::getNormalizedDebt()`, reporting incorrect values.

Consider only updating the `reserve.usageIndex` when there are active borrows in the system.

[L-03] Protocol tokens do not validate scaled amounts

The `RToken`, `DebtToken`, and `DeToken` do not validate the calculated scaled amount when minting and burning. For example:

```
// RToken.sol
function mint(
    address caller,
    address onBehalfOf,
    uint256 amount,
    uint256 index
) external override onlyLendingPool returns (uint256, uint256, uint256) {
```



```
...
    _rawTotalDeposits += amount.rayDiv(index);

    _mint(onBehalfOf, amount.toUint128());
...

function burn(
    address from,
    address receiverOfUnderlying,
    uint256 amount,
    uint256 index
) external override onlyLendingPool returns (uint256, uint256, uint256) {
...
    uint256 rawAmount = amount.rayDiv(index);
    if(rawAmount > _rawTotalDeposits) {
        _rawTotalDeposits = 0;
    } else {
        _rawTotalDeposits -= rawAmount;
    }

    _burn(from, amount.toUint128());
}
```

```
// DebtToken.sol
function mint(
    address caller,
    address onBehalfOf,
    uint256 amount,
    uint256 poolUsageIndex,
    bytes calldata asset
) external onlyLendingPool returns (bool, uint256, uint256, uint256) {
...
    _mint(onBehalfOf, amount.toUint128());

    _rawTotalBorrows += amount.rayDiv(
        ILendingPool(_lendingPool).getNormalizedDebt()
    );
}
```

We will notice above that the scaled amount is used to update the `_rawTotalDeposits` / `_rawTotalBorrows` state variables, but the unscaled `amount` is used for the `mint` / `burn`. This means that if the `amount.rayDiv(index)` calculation rounds down to 0, the `_rawTotalBorrows` / `_rawTotalDeposits` state variables will remain unchanged, despite the fact that the actual total supply of the token in question (and the user's balance) has been updated. This results in the internally tracked global state deviating from the actual state of the tokens.

This deviation can result in the raw and scaled total supplies (i.e. `RToken::totalSupply` / `RToken::getRawTotalDeposits`) of the tokens returning inaccurate values.

Note that this is only possible to occur if the `minDepositAmount` is set to `0` and if the index is large, i.e. `> 2e27`.

Consider requiring the calculated scaled amounts to be `> 0`.



[L-04] Burned RAACNFT tokens can be re-minted

When users mint RAACNFTs they can supply, any tokenId that is tied to a valid house price:

```
function mint(uint256 _tokenId, uint256 _amount) public override nonReentrant
notBlacklisted(msg.sender) {
...
    (uint256 price, uint256 timestamp) = raac_hp.getLatestPrice(_tokenId);
    if(price == 0) revert HousePriceZero();
    _validatePriceFreshness(timestamp);
...
    // mint tokenId to user
    _initiateMint(msg.sender, _tokenId, price);
...
}
```

Additionally, tokens can be burned by any user (if they are owner) or admin (can burn any token):

```
function burn(uint256 tokenId) external {
    if (!allowBurning) revert BurningDisabled();

    // Only owner or admin can burn
    address tokenOwner = ownerOf(tokenId);
    bool isAdmin = hasRole(ADMIN_ROLE, msg.sender);

    if (!isAdmin && tokenOwner != msg.sender) {
        revert InvalidAddress();
    }

    _burn(tokenId);
}
```

This means that admin, or the user (owner), can burn a specific tokenId and then another user can immediately re-mint that same tokenId by supplying the tokenId to the mint function. This is possible because the underlying ERC721 logic does not restrict re-minting tokenIds and only requires that the previous owner of the token is address(0) for mints and != address(0) for burns:

```
function _update(address to, uint256 tokenId, address auth) internal virtual returns
(address) {
    address from = _ownerOf(tokenId);

    // Perform (optional) operator check
    if (auth != address(0)) {
        _checkAuthorized(from, auth, tokenId);
    }

    // Execute the update
    if (from != address(0)) {
        // Clear approval. No need to re-authorize or emit the Approval event
        _approve(address(0), tokenId, address(0), false);

        unchecked {
```



```
        _balances[from] -= 1;
    }
}

if (to != address(0)) {
    unchecked {
        _balances[to] += 1;
    }
}

_owners[tokenId] = to;

emit Transfer(from, to, tokenId);

return from;
}

...

function _mint(address to, uint256 tokenId) internal {
    if (to == address(0)) {
        revert ERC721InvalidReceiver(address(0));
    }
    address previousOwner = _update(to, tokenId, address(0));
    if (previousOwner != address(0)) {
        revert ERC721InvalidSender(address(0));
    }
}

...

function _burn(uint256 tokenId) internal {
    address previousOwner = _update(address(0), tokenId, address(0));
    if (previousOwner == address(0)) {
        revert ERC721NonexistentToken(tokenId);
    }
}

}
```

Consider tracking a global `nextTokenId` state variable that increases with each mint. This would guarantee that each new token minted will be unique, and old `tokenIds` can not be reused. Alternatively, consider introducing a `burned[tokenId]` mapping to track destroyed tokens and thus restrict them from being minted again.

[L-05] Liquidations fail when `LendingPool` is paused

Liquidations will succeed when the `RWAVault` is paused, since there exists a dedicated `RWAVault::poolDepositAsset` function that allows the stability pool to deposit the seized asset into the vault when it is paused. However, during the liquidation process, the stability pool will also attempt to withdraw and/or deposit into the `LendingPool` :

```
function liquidateBorrower(address poolAdapter, address vaultAdapter, address user, bytes
calldata data, uint256 minSharesOut) external onlyProxy {
    ...

    // We unwind the position
```



```
if (rTokenAmountRequired > 0) {
    lendingPool.withdraw(rTokenAmountRequired);
}

crvUSDToken.safeTransfer(address(rToken), scaledPositionDebt);

// Deposit crvUSD back to get rTokens (including the excess)
// Get the final crvUSD balance after the exchange
uint256 finalCRVUSDBalance = crvUSDToken.balanceOf(address(this));
if (finalCRVUSDBalance > 0) {
    // Approve lending pool to take crvUSD for deposit
    bool approveCRVUSDDeposit = crvUSDToken.approve(address(lendingPool),
finalCRVUSDBalance);
    if (!approveCRVUSDDeposit) revert ApprovalFailed();
    lendingPool.deposit(finalCRVUSDBalance);
}
```

If the `LendingPool` is paused, then the liquidation call will fail, which can lead to the unhealthy position becoming more under-collateralized as interest accrues in this paused state. Additionally, the `LendingPool::finalizeLiquidation` function is not guarded with a `whenNotPaused` modifier, which suggests this flow should not be affected by a paused state.

Note that the `LendingPool` can also specifically have withdrawals paused via the `Parameters.withdrawalsPaused` flag. However, the pool can also be paused as a whole via `LendingPool::pause`.

Consider introducing privileged `deposit` and `withdraw` functions that would allow the stability pool to interact with the `LendingPool` when it is paused, similar to how the `RWAVault::poolDepositAsset` function is meant to be utilized.

[L-06] `RToken` does not include interest in `Mint` event emission

When a user mints `RTokens`, the function will mint the requested principal `amount` and any pending interest:

```
function mint(
    address caller,
    address onBehalfOf,
    uint256 amount,
    uint256 index
) external override onlyLendingPool returns (uint256, uint256, uint256) {
...
    // Will update user index
    _mintUserInterest(onBehalfOf, index);
...
    _mint(onBehalfOf, amount.toUint128());

    emit Mint(caller, onBehalfOf, amount, index);
...

function _mintUserInterest(address user, uint256 currentPoolIndex) internal {
...
}
```



```
// If scaled balance is greater than user balance, mint the difference
if (scaledBalance > userBalance) {
    uint256 userIncrease = scaledBalance - userBalance;
    _mint(user, userIncrease);
}
```

However, we will notice above that the `Mint` event will only capture the principal `amount` minted and not the interest. Conversely, we will notice that the `DebtToken` correctly captures the interest minted in its `Mint` event:

```
function mint(
    address caller,
    address onBehalfOf,
    uint256 amount,
    uint256 poolUsageIndex,
    bytes calldata asset
) external onlyLendingPool returns (bool, uint256, uint256, uint256) {
    ...
    // Update raw total borrows with only the new borrow amount (not the interest)
    uint256 userIncrease = _mintUserInterest(onBehalfOf,
    ILendingPool(_lendingPool).getPositionDebt(adapter, onBehalfOf, data),
    ILendingPool(_lendingPool).getPositionScaledDebt(adapter, onBehalfOf, data));
    _mint(onBehalfOf, amount.toUint128());

    _rawTotalBorrows += amount.rayDiv(
        ILendingPool(_lendingPool).getNormalizedDebt()
    );

    emit Mint(caller, onBehalfOf, amount + userIncrease, 0, poolUsageIndex, positionIndex);
}
```

As a result, mints for the `RToken` can emit inaccurate events, which can potentially affect off-chain observability.

Consider including the accrued interest in the `RToken::Mint` event emission.

[L-07] Prime rate change limits are too permissive and prevent decreases below 5%

The `setPrimeRate()` function allows a maximum change of 5% (500 basis points) per update, which is significantly higher than historical market movements. The maximum single-step change in the US Prime Rate over the last 30 years has been approximately 1%, suggesting the limit should be around 1.5% to better reflect real-world volatility and properly guard against unreasonable changes.

Additionally, the decrease limit calculation prevents the prime rate from being reduced when it's already below 5%. This is problematic because the US Prime Rate has historically been below 5% (e.g., during 2008-2015 and 2020-2022), and preventing rate decreases in such conditions would make the protocol's borrowing rates uncompetitive compared to other protocols, leading to lower utilization.



Reduce the change limit from 5% to 1.5% to better align with historical volatility, and fix the decrease calculation to allow reductions below 5% by using a fixed percentage decrease rather than clamping to zero.

[L-08] `LendingPool.updateState()` does not recalculate interest rates after accrual

The `updateState()` function in `LendingPool` only calls `updateReserveState()`, which updates the liquidity and usage indices to accrue interest over time. However, it does not call `updateInterestRatesAndLiquidity()` to recalculate the current interest rates based on the updated utilization.

When interest accrues, the total debt increases (as interest is effectively "minted" into the debt token), which changes the utilization rate (`totalUsage / totalLiquidity`). Since interest rates are calculated based on utilization, the rates become stale after `updateState()` is called. The rates will only be updated on the next operation that calls `updateInterestRatesAndLiquidity()` (such as `deposit()`, `withdraw()`, `borrow()`, or `repay()`).

This means that if `updateState()` is called to sync the pool state (e.g., by an off-chain keeper or frontend), the displayed or calculated interest rates will be inaccurate until the next user operation. While this doesn't directly cause loss of funds, it creates a discrepancy between the actual accrued interest and the rates used for calculations.

Add a call to `ReserveLibrary.updateInterestRatesAndLiquidity(reserve, rateData, 0, 0)` after `updateReserveState()` in `updateState()` to ensure interest rates are recalculated based on the updated utilization after interest accrual.

[L-09] Missing event emission for key parameter changes

Throughout the protocol some functions that change key parameters do not emit an according event. Consider adding an event emission to them.

- `NFTRoyaltyFeeCollector.setFeeCollector()`.
- `ERC20VaultAdapter.setPriceOracle()`.
- `ERC721VaultAdapter.setPriceOracle()`.

[L-10] Liquidity buffer ratio must initialize to zero as per docs

The `liquidityBufferRatio` parameter is initialized to `20_00` (20%) in the constructor, but according to documentation it should be initialized to `0`. This requires an additional call to `setParameter()` to set it to `0` after deployment, which is inefficient and unnecessary. Consider setting the `liquidityBufferRatio` to 0 in the constructor.



[L-11] Min price `Threshold` lets NFT minting at discount during depeg

The `minPriceThreshold` in `CrvUSDToUSDOracle` artificially inflates the value of crvUSD during depeg events, allowing users to mint NFTs at a discount. When crvUSD depegs below \$0.90, the `_clampCircuitBreaker()` function `minPriceThreshold` (0.9e18) instead of the actual depegged price. This inflated price is then used by `RAACHousePrices.getLatestPrice()` to calculate the house price in crvUSD, resulting in users paying less crvUSD than they should.

For example, if a house is valued at \$100,000 USD and crvUSD depegs to \$0.80, the oracle will report \$0.90 due to the minimum threshold. Users would pay approximately 111,111 crvUSD ($\$100,000 / \0.90) instead of the correct 125,000 crvUSD ($\$100,000 / \0.80), receiving a discount of about 11%. This economic advantage can be exploited during market stress when crvUSD depegs, allowing arbitrageurs to mint NFTs cheaply and potentially profit from the protocol's loss.

Recommendation

Remove the `minPriceThreshold` from the circuit breaker logic. The maximum threshold (`maxPriceThreshold`) provides sufficient protection against price manipulation upward, while the minimum threshold creates an exploitable vulnerability during depeg events.